



POPACheck: A Model Checker for Probabilistic Pushdown Automata

Francesco Pontiggia^(✉), Ezio Bartocci, and Michele Chiari



TU Wien, Vienna, Austria
{francesco.pontiggia,ezio.bartocci,
michele.chiari}@tuwien.ac.at



Abstract. We present **POPACheck**, the first model checking tool for probabilistic Pushdown Automata (pPDA) supporting temporal logic specifications. **POPACheck** provides a user-friendly probabilistic modeling language with recursion that automatically translates into Probabilistic Operator Precedence Automata (pOPA). pOPA are a class of pPDA that can express all the behaviors of probabilistic programs: sampling, conditioning, recursive procedures, and nested inference queries. On pOPA, **POPACheck** can solve reachability queries as well as qualitative and quantitative model checking queries for specifications in Linear Temporal Logic (LTL) and a fragment of Precedence Oriented Temporal Logic (POTL), a logic for context-free properties such as pre/post-conditioning.

Keywords: Probabilistic Model Checking · Pushdown Model Checking · Temporal Logic · Operator Precedence Languages

1 Introduction

The last two decades saw great efforts towards the analysis of probabilistic Pushdown Automata (pPDA) [3–6,14,31], and the equivalent model of Recursive Markov Chains (RMCs) [16–20,54] as a succinct formalism to express infinite-state probabilistic systems and to model probabilistic programs with nested and possibly recursive procedures. However, no existing tool implements the presented temporal logic model checking algorithms, due to their computational complexity and many practical obstacles. With

POPACheck, we leverage various recent results on pPDA analysis to produce an efficient model checking tool that scales beyond toy examples, and forms a baseline for future verification tools for infinite-state probabilistic systems.

Illustrative Example. Consider the Sherwood [34] variant of the binary search (Fig. 1), a well-known recursive algorithm. **B()** searches for **val** in the array **a[left..right]**. Unlike the deterministic version, in each iteration **B()** selects

```
# global variables: a[], mid, val
B(u4 left, u4 right) {
  mid = Uniform(left, right);
  if (left < right){
    if (a[mid] < val) {
      left = min(mid + 1, right);
      B(left, right);
    } else { if (a[mid] > val) {
      right = max(mid - 1, left);
      B(left, right);
    } }
  }
}
```

Fig. 1. Sherwood Binary search.

the pivot `mid` randomly among the remaining portion of `a`. Thanks to randomness, worst-, best- and average runtime align. If `a` has a finite bound and its elements have a finite domain, `B()` can be translated into a finite-state pPDA: automaton states model global and local variable values, while stack symbols model procedure parameter values. A relevant property for this program is **partial correctness** [37]: when `B()` is invoked in a state where `left ≤ right`, `a[left..right]` is sorted and `val` occurs in `a[left..right]`, then at return `mid` stores the index of `a` where `val` lies. This property, just like pre/post-conditioning, requires to match each call to `B()` in the recursion to its corresponding return, and skip inner calls to `B()` in the execution trace. It is a context-free property beyond LTL’s expressivity, limited to regular properties. POTL [9] is an expressive logic based on Operator Precedence Languages (OPLs) [22], a subclass of context-free languages, and expresses partial correctness as:

$$\Box(\text{call} \wedge B \wedge \text{sorted} \wedge \text{valOccurs} \wedge \text{left} \leq \text{right} \implies \chi_F^u \text{a[mid]} == \text{val})$$

Probabilistic Programming. Probabilistic programs have recently gained popularity in AI and machine learning [23], where they are employed as succinct models for Bayesian inference [26]. In addition to randomized assignment, a prominent feature of this programming model is *conditioning*, which allows for adding evidence of observed events by conditioning the program variables to take certain values. When using rejection sampling, a probabilistic program contains statements `observe(e)`, where `e` is a Boolean condition. If `e` is not satisfied, the current execution trace is rejected: in such a case, we assume that the program is restarted. The semantics of a probabilistic program is the probability distribution in the return statement, also called *posterior*. Probabilistic programs can also be nested: a program samples a value from a distribution represented by another probabilistic program. They are known under the name of *nested queries* [24], and model scenarios beyond the expressivity of flat probabilistic programs: most notably, metareasoning patterns (the so called Theory of Mind) [48,55], linguistics [45], cognitive science [25], multi-agent planning [46] and sequential decision making [21]. To the best of our knowledge, pOPA [40] are the smallest subclass of pPDA that can model effectively these behaviors. OPA traces are OPLs. Unlike general context-free languages, OPLs are closed under most relevant Boolean operations [33]. Let $\mathcal{B}$ be an (deterministic or separated) Operator Precedence Büchi Automaton (ω OPBA), the class of automata recognizing OPLs. When a pOPA $\mathcal{A}$ is defined over the same alphabet as $\mathcal{B}$, we can verify automatically $\mathcal{A}$ against $\mathcal{B}$ via automata-based model checking [40].

Our Approach. The main challenge in pPDA model checking is computing the *termination probabilities*, which involves solving a system of **non-linear** equations. While tools like PRemo [53] approximate them from below, for general model checking we need to know whether these quantities sum up to exactly 1. We could employ off-the-shelf SMT solvers (e.g. Z3 [35]) with decision procedures for the Existential first-order Theory of the Reals (ETR) (QF_NRA), but they offer only doubly exponential decision procedures [30], although ETR is in PSPACE [7,43]. We thus devise a semi-algorithm leveraging both certificates for termination probabilities [51] via a numeric method called Optimistic Value

Iteration (OVI), and certificates for expected runtimes [52]. A second challenge is the need for deterministic automata (or weak variants thereof) in probabilistic verification—a common problem already occurring in the far simpler setting of LTL model checking for Markov Chains [1, 2, 11]. In this regard, we exploit [19, 40] which provide resp. single exponential model checking algorithms (i.e., avoiding determinization) for LTL and a fragment of POTL (POTL $\text{f}\chi$). These algorithms represent the theoretical ground of POPACheck.

POPACheck is hosted on Github at <https://github.com/michiari/POMC/>. It is an extension of the POMC tool [8, 39, 41], and relies on its modules for constructing automata from formulae.

Contributions. We present i) a semi-algorithm overcoming numerical issues regarding computing termination probabilities; ii) POPACheck, a model checker for pPDA based on this semi-algorithm; iii) a user-friendly domain-specific language for recursive probabilistic programs; and iv) an extensive experimental evaluation with a benchmark of programs and LTL/POTL $\text{f}\chi$ formulae.

2 Background

2.1 Probabilistic Operator Precedence Automata (pOPA)

pOPA are pPDA with state labels from a set Σ . State labels drive the stack behavior of pOPA through three *Precedence Relations (PRs)*: given two labels a and b , we say a *yields precedence* to b iff $a < b$, a and b are *equal in precedence* iff $a \doteq b$, and a *takes precedence* from b iff $a > b$. An Operator Precedence Matrix (OPM) is a total function $M : \Sigma^2 \rightarrow \{<, \doteq, >\}$; the special symbol $\#$ is s.t. $\# < a$ for all $a \in \Sigma$. In the following, let $\mathfrak{D}(S) = \{f : S \rightarrow [0, 1] \mid \sum_{s \in S} f(s) = 1\}$ denote the set of probability distributions on a finite set S .

Definition 1 ([40]). *A pOPA is a tuple $\mathcal{A} = (\Sigma, M, Q, u_0, \delta, \Lambda)$ where: Σ is a finite set of state labels; M is an OPM; Q is a finite set of states; u_0 is the initial state; $\Lambda : Q \rightarrow \Sigma$ is a state labeling function; and δ is a triple of transition functions $\delta_{\text{push}} : Q \rightarrow \mathfrak{D}(Q)$, $\delta_{\text{shift}} : Q \rightarrow \mathfrak{D}(Q)$, and $\delta_{\text{pop}} : (Q \times Q) \rightarrow \mathfrak{D}(Q)$, such that pop moves have the following condition, for all $u, s, v \in Q$:*

$$\delta_{\text{pop}}(u, s)(v) > 0 \implies \forall a \in \Sigma : a > \Lambda(u) \implies a > \Lambda(v).$$

The semantics of $\mathcal{A}$ is an infinite Markov chain [1] $\Delta(\mathcal{A})$ with vertex set $Q \times (\Gamma^* \{\perp\})$ where $\perp$ is the initial stack symbol, which can never be pushed or popped, and $\Gamma = \Sigma \times Q$ is the set of stack symbols. PRs decide whether to push onto the stack, update the topmost symbol, or pop from it. For any stack contents $A \in \Gamma^* \{\perp\}$:

push: $(u, A) \xrightarrow{x} (v, [\Lambda(u), u]A)$ if $\text{smb}(A) < \Lambda(u)$ and $\delta_{\text{push}}(u)(v) = x$;
shift: $(u, [a, s]A) \xrightarrow{x} (v, [\Lambda(u), s]A)$ if $a \doteq \Lambda(u)$ and $\delta_{\text{shift}}(u)(v) = x$;
pop: $(u, [a, s]A) \xrightarrow{x} (v, A)$ if $a > \Lambda(u)$ and $\delta_{\text{pop}}(u, s)(v) = x$;

where $\text{smb}(\perp) = \#$ and $\text{smb}([a, r]) = a$ for $[a, r] \in \Gamma$.

Stack symbols are pairs of a label and a state, and the PR between the label in the topmost stack symbol and that of the current state decides the next move.

If such PR is $\leq$, a push move puts the current state and its label onto the stack. Since $\text{smb}(\perp) = \#$, the first move is always a push. If the PR is $\dot{=}$, a shift move updates the topmost stack symbol by replacing its label with the current state's. Finally, if the PR is $\geq$, the topmost stack symbol gets popped.

A run of $\mathcal{A}$ is a path in $\Delta(\mathcal{A})$ starting in $(u_0, \perp)$. The probability space on the set of runs is obtained by the cylinder set construction as for Markov chains [1]. The set of infinite words formed by labels of states in a run is an OPL [40].

PRs guide the stack behavior of pOPA, and PRs between state labels completely determine whether the pOPA pushes, updates, or pops a stack symbol. This dependency of the stack behavior on labels, and hence traces, enables the definition of stack-aware context-free modalities in the specification formalism (POTL), by allowing the definition of a synchronized product between pOPA and a pushdown automaton encoding the specification (cf. [40]). We thus define state labels to describe events that affect the program stack, and we define PRs between state labels so that such events have the same effect on the pOPA stack that they have on the stack of activation frames in a high-level procedural probabilistic programming language with rejection sampling.¹

State labels in OPM M_{call} (Fig. 2) represent traces of probabilistic programs. **call** and **ret** are function calls and returns, **stm** statements that do not affect the stack (e.g., randomized assignments). **qry** is the *conditional sampling* operator (roughly corresponding to **query** in Church [24]): it reifies the posterior distribution of a probabilistic program, allowing for sampling from it. **obs** denotes an unsatisfied observation. The table is to be read by row—e.g., we have $\text{call} \dot{=} \text{ret}$ and $\text{ret} \geq \text{call}$. PRs are such that a pOPA always pushes from a **call** state ($\text{call} \leq \text{call}$, etc.), and pops after being in a **ret** state through a shift move ($\text{call} \dot{=} \text{ret}$ and **ret** takes precedence from other symbols). This way, the pOPA stack mimics the program's stack. **stm** causes a push immediately followed by a pop (because $\text{stm} \geq$ all labels), thus leaving the stack unchanged. Moreover, **obs** triggers pop moves that unwind the stack until they reach the first **qry** symbol. Thereby, the pOPA excludes (or rejects) the trace at hand from the posterior distribution of the current probabilistic program without affecting outer queries, effectively simulating nested rejection sampling [36].

	call	ret	qry	obs	stm
call	$\leq$	$\dot{=}$	$\leq$	$\geq$	$\leq$
ret	$\geq$	$\geq$	$\geq$	$\geq$	$\geq$
qry	$\leq$	$\dot{=}$	$\leq$	$\leq$	$\leq$
obs	$\geq$	$\geq$	$\geq$	$\geq$	$\geq$
stm	$\geq$	$\geq$	$\geq$	$\geq$	$\geq$

Fig. 2. OPM M_{call} .

2.2 Specification Formalism

We express specifications through a temporal logic with the following syntax:

$$\varphi := a \mid T \mid \neg\varphi \mid \varphi \vee \varphi \mid \bigcirc\varphi \mid \varphi \mathcal{U} \varphi \mid \bigcirc^t\varphi \mid \chi_F^t\varphi \mid \varphi \mathcal{U}_\chi^t\varphi$$

Formulae are evaluated in the first position of program traces, and subformulae in further positions. a is an *atomic proposition* from a finite set AP containing

¹ Note that PRs are not ordering relations, so they need not enjoy standard properties such as reflexivity and transitivity.

labels from Fig. 2 and names of program functions. T is any term of the form $e_1 \bowtie e_2$, where $\bowtie$ is a binary comparison operator ($=$, $>$, etc.) and e_1, e_2 are integer arithmetic expressions involving program variables and constants. $\neg$ and $\vee$ are propositional operators, to which we add the derived operators $\wedge$ and $\implies$ with the usual semantics. $\bigcirc \varphi$ and $\varphi_1 \mathcal{U} \varphi_2$ are the *next* and *until* operators from LTL [38], resp. meaning that φ will hold in the next time instant, and that φ_1 holds until a time instant in which φ_2 does. We use the derived operators *eventually* $\Diamond \varphi \equiv \top \mathcal{U} \varphi$, and *globally* $\Box \varphi \equiv \neg \Diamond \neg \varphi$.

The remaining operators form a fragment of POTL called POTLf $_{\chi}$. They move up ($t = u$) and down ($t = d$) among function frames in the program stack. $\bigcirc^d \varphi$ holds if the next time instant belongs to the same or a lower frame and φ holds in it, and *vice versa* for $\bigcirc^u$. χ_F^d moves along a binary relation that links function calls to their returns and inner function calls.

In the program of Fig. 3, the relation links events representing **calls** to $\mathbf{f1}()$ to those representing the inner **call** to $\mathbf{f2}()$ and $\mathbf{f1}$'s return statement. E.g., if evaluated in an instant labeled with a **call** to $\mathbf{f1}$, $\chi_F^d(\mathbf{call} \wedge \mathbf{x} > 0)$ holds if $\mathbf{x} > 0$ in the instant when $\mathbf{f1}$ calls $\mathbf{f2}$, and $\chi_F^d(\mathbf{ret} \wedge \mathbf{y} > 0)$ holds if $\mathbf{f1}$ returns in an instant in which $\mathbf{y} > 0$. Moreover, $\chi_F^u \mathbf{obs}$ holds if a false **observe** is encountered anytime during the execution of $\mathbf{f1}$. The semantics of $\mathcal{U}_{\chi}^t$ are the same as the LTL until, except it works on paths obtained by iterating the $\bigcirc^t$ and χ_F^t operators, for $t \in \{u, d\}$. For a complete definition, cf. [10].

```

f1() {
  ... // other statements
  f2(x);
  ...
  return y;
}

```

Fig. 3. Program stub.

3 Model Checking Algorithm

3.1 The Support Chain

Let $\mathcal{A} = (\Sigma, M, Q, u_0, \delta, \Lambda)$ be a pOPA. A *support* is a sequence of pOPA moves $u_0 \rightarrow u_1 \dashrightarrow \dots \dashrightarrow u_{\ell} \xrightarrow{u_0} u_{\ell+1}$, denoted as $u_0 \leadsto u_{\ell+1}$, that occur from the move pushing a stack symbol $[A(u_0), u_0]$ to the move popping it. Given $u, v \in Q$ and $\alpha \in \Gamma$, we define $\llbracket u, \alpha \mid v \rrbracket$ as the probability that $\mathcal{A}$ starting in $(u, \alpha \perp)$ reaches v at the end of the support that put α on top of the stack. Such *termination probabilities* [4, 18] are the least non-negative solutions of the equation system $\mathbf{v} = f(\mathbf{v})$, where $\mathbf{v}$ is the vector of triples $\llbracket u, \alpha \mid v \rrbracket$ for all $u, v \in Q$, $\alpha \in \Gamma$ and $f(\llbracket u, \alpha \mid v \rrbracket)$ is given by

$$\begin{cases} \sum_{r,t \in Q} \delta_{push}(u)(r) \llbracket r, [\Lambda(u), u] \mid t \rrbracket \llbracket t, \alpha \mid v \rrbracket & \text{if } \alpha = \perp, \text{ or } \alpha = [a, s] \text{ and } a \triangleleft \Lambda(u) \\ \sum_{r \in Q} \delta_{shift}(u)(r) \llbracket r, [\Lambda(u), s] \mid v \rrbracket & \text{if } \alpha = [a, s] \text{ and } a \doteq \Lambda(u) \\ \delta_{pop}(u, s)(v) & \text{if } \alpha = [a, s] \text{ and } a \triangleright \Lambda(u) \end{cases}$$

The probability that $\alpha \in \Gamma$ is never popped after a run visits a configuration $(u, \alpha A)$ for $u \in Q$ and some stack contents A is $\llbracket u, \alpha \uparrow \rrbracket = 1 - \sum_{v \in Q} \llbracket u, \alpha \mid v \rrbracket$.

The *support chain* is a finite Markov chain that replaces supports with single transitions, and describes the behavior of pOPA runs while preserving their probability distribution.

Definition 2 ([40]). *The support chain $M_{\mathcal{A}}$ of $\mathcal{A}$ is a Markov chain with states in $\mathcal{C} = \{(u, \alpha) \in Q \times \Gamma_{\perp} \mid \llbracket u, \alpha \uparrow \rrbracket > 0\}$, initial state $(u_0, \perp)$, and a transition probability function $\delta_{M_{\mathcal{A}}}$ such that*

- $\delta_{M_{\mathcal{A}}}(u, [a, s])(v, [\Lambda(u), s]) = \delta_{shift}(u)(v) \llbracket v, [\Lambda(u), s] \uparrow \rrbracket / \llbracket u, [a, s] \uparrow \rrbracket$
for all $(u, [a, s]), (v, [\Lambda(u), s]) \in \mathcal{C}$ such that $a \doteq \Lambda(u)$;
- otherwise, for all $(u, \alpha), (v, \alpha') \in \mathcal{C}$,
 $\delta_{M_{\mathcal{A}}}(u, \alpha)(v, \alpha') = (P_{push} + P_{supp}) \llbracket v, \alpha' \uparrow \rrbracket / \llbracket u, \alpha \uparrow \rrbracket$ where
 - $P_{push} = \delta_{push}(u)(v)$ if $smb(\alpha) \triangleleft \Lambda(u)$, and $P_{push} = 0$ otherwise;
 - $P_{supp} = \sum_{v' \in Q} \delta_{push}(u)(v') \llbracket v', [\Lambda(u), u] \mid v \rrbracket$, if $\alpha = \alpha'$, $smb(\alpha) \triangleleft \Lambda(u)$, and $\mathcal{A}$ has a support $u \rightsquigarrow v$, and $P_{supp} = 0$ otherwise.

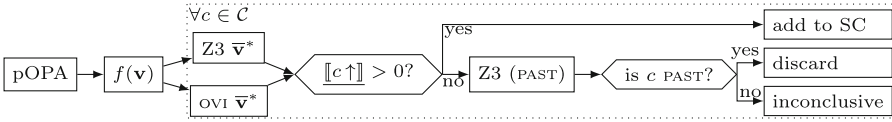


Fig. 4. Overview of the semi-algorithm for building the support chain (SC).

During infinite runs of $\mathcal{A}$, the stack contains symbols that are never popped. States of $M_{\mathcal{A}}$ consist of such symbols, and the current state of $\mathcal{A}$ when they are put on the stack. $M_{\mathcal{A}}$ has three types of transitions: push and shift moves that put on the stack (resp. update) a symbol that will never be popped, and *support* edges, which summarize the finite portion of a run from when a stack symbol is pushed to when it is popped—a *support*. Probabilities assigned to edges $c \rightarrow c'$ are conditioned on the fact that the stack symbol in the target node c' is never popped, given that the one in the starting node c is also never popped: hence all probabilities are multiplied by $\llbracket c' \uparrow \rrbracket / \llbracket c \uparrow \rrbracket$. Probabilities for push and shift edges follow directly from $\mathcal{A}$'s δ distributions, while those for support edges include P_{supp} , the probability that the stack symbol underlying the edge is popped. Thus, $M_{\mathcal{A}}$ describes the limit behavior of $\mathcal{A}$ by only considering permanent stack symbols and transitions that created them, while summarizing with single edges run portions that involve stack symbols that are eventually popped.

To build the support chain of $\mathcal{A}$, we must check whether $\llbracket c \uparrow \rrbracket > 0$ for all $c \in Q \times \Gamma_{\perp}$. We propose the approach sketched in Fig. 4. We further optimize this algorithm by applying all steps bottom-up to Strongly Connected Components (SCCs) of the graph of nodes c .

Let $\mathbf{v}^*$ be the *least* solution of $\mathbf{v} = f(\mathbf{v})$, i.e., the vector of termination probabilities $\llbracket c \mid v \rrbracket$. Recall that $\llbracket c \uparrow \rrbracket = 1 - \sum_{v \in Q} \llbracket c \mid v \rrbracket$. First, we find an upper bound $\bar{\mathbf{v}}^*$ for $\mathbf{v}^*$, which we use to compute a lower bound $\llbracket c \uparrow \rrbracket$ for $\llbracket c \uparrow \rrbracket$. If $\llbracket c \uparrow \rrbracket > 0$, then $\llbracket c \uparrow \rrbracket > 0$ and we add c to the support chain. Otherwise, we make the hypothesis that $\llbracket c \uparrow \rrbracket = 0$, i.e., that c is almost-surely terminating (AST), and check the stronger condition that c is positively AST (PAST). A node $c = (u, \alpha)$ is PAST if stack symbol α is popped with probability 1 in *finite expected time*. We check PAST by solving with Z3 a system of linear equations [52] for the expected termination times. If a solution exists, then c is PAST and therefore AST: we do not add c to the support chain. Otherwise, the whole algorithm is inconclusive.

We find the upper bound $\bar{\mathbf{v}}^*$ in two different ways. In both cases, we first find a lower bound $\underline{\mathbf{v}}^*$ to $\mathbf{v}^*$ via a decomposed variant of Newton’s method [12, 18], after cleaning the system from zero-solution variables via value iteration.

- In the first method, we give $\underline{\mathbf{v}}^*$ as a hint to Z3, and obtain $\bar{\mathbf{v}}^*$ as a model for $\mathbf{v}$ in the query $\underline{\mathbf{v}}^* \leq \mathbf{v} \leq \underline{\mathbf{v}}^* + \varepsilon \wedge \mathbf{v} \geq f(\mathbf{v})$, for a small positive ε .
- The other method employs OVI [51], which computes $\bar{\mathbf{v}}^*$ numerically. We run OVI initialized on $\underline{\mathbf{v}}^*$.

3.2 Avoid Determinization with a Separation-Based Approach

A prominent approach to LTL model checking on probabilistic systems expresses specifications as deterministic automata. Storm [28] and PRISM [32] translate the input formula into a Deterministic Rabin Automaton (DRA) [1], with a worst-case doubly exponential blowup. While singly exponential algorithms that avoid determinization exist [2, 11], a common argument for using the conceptually simpler notion of Rabin automata is the existence in practice of effective translations from LTL formulae to relatively small-sized DRA [13]. Unfortunately, the same cannot be said for POTL formulae: the automata construction for POTL is much more involved [8], and easily generates intractable automata.

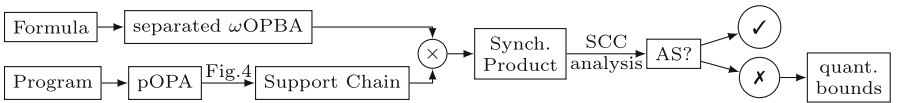


Fig. 5. Overview of model checking probabilistic programs against POTLfx formulae.

POPACheck takes a different approach, outlined in Fig. 5. LTL formulae can be translated into *separated* Büchi automata, i.e., such that the languages they accept starting from different states are disjoint [11]. Similarly, POTLfx formulae

can be translated into ω OPBA, i.e., pushdown automata that capture the class of infinite-word OPLs, that are *separated*. Thanks to separation, we can build a synchronized product between the support chain of the pOPA modeling the program and the automaton encoding the formula. We then perform qualitative model checking by analyzing the SCCs of the product. Since formulae are translated into automata of exponential size [8, 49], the whole procedure requires time exponential in formula size and space polynomial in pOPA size. For quantitative model checking, we compute numerically the probabilities associated with edges in the synchronized product: edges subsuming supports require solving equation systems resembling those for termination probabilities. We compute bounds for them with OVI. Since the product has size exponential in formula length, quantitative model checking is in EXPSPACE. Cf. [40] for details.

4 Input Language

POPACheck analyzes programs written in MiniProb, a simple probabilistic programming language (Fig. 6). MiniProb supports (un)signed integer variables of arbitrary width (u8 is an 8-bit unsigned type) and fixed-size arrays. Functions take parameters by value or value-result (with $\&$). Actual parameters can only be variable identifiers for value-result parameters, and any expression if passed by value. Expressions consist of variables, array indexing, integer constants,

and the usual arithmetic and

Boolean operators, including comparisons. Boolean operators handle integers (0 means false, everything else true). Programs may sample from **Bernoulli**(e_1, e_2), which returns 1 with probability $p = e_1/e_2$, and 0 with probability $1 - p$, or from **Uniform**(e_1, e_2), which samples uniformly among integers from e_1 to $e_2 - 1$. Random assignments of the form $x = e_1\{e_2/e_3\}e_4$ mean that x is assigned the value of e_1 with probability e_2/e_3 , and e_4 with probability $1 - e_2/e_3$. Finally, functions can **query** the distribution on value-result parameters of another function, and condition on a Boolean expression with **observe**.

```

prog := [decl; ...] func [func ...]
decl := type identifier [, identifier ...]
type := bool | uint | sint | uint[int] | sint[int]
func := f (type [&]x1 [, type [&]x2 ...])
      { [decl; ...] block }
stmt := lval = e
      | lval = Distribution(...)
      | lval = e1{e2 : e3}[e4{e5 : e6}...]en
      | [query] f (e1 | lval1 [, e2 | lval2 ...])
      | if (e) {block} else {block}
      | while (e) {block}
      | observe (e)
block := stmt; [stmt ...];
lval := identifier | identifier[e]

```

Fig. 6. MiniProb syntax.

Table 1. Benchmark formulae.

#	Formula	#	Formula
Q.1	$\Diamond \Box (\neg \text{obs})$	Q.2	$\Box (\text{qry} \implies \bigcirc^d (\text{call} \wedge \neg \chi_F^u \text{obs}) \vee \chi_F^d (\text{call} \wedge \neg \chi_F^u \text{obs}))$
Q.3	$\Box (\text{call} \wedge \text{Alice} \wedge p \geq 0.4 \implies \neg \chi_F^u \text{obs})$	Q.4	$\neg \text{sampleA} \mathcal{U} (\text{call} \wedge \text{sampleA} \wedge \chi_F^d (\text{qry} \wedge \bigcirc^d (\text{call} \wedge \text{sampleA})))$
Q.5	$\neg \Diamond (\text{qry} \wedge \chi_F^u (\text{sampleA} \wedge \text{opR} == 0 \wedge \text{opC} == 1))$	Q.6	$(\neg \text{elder}) \mathcal{U}_\chi^d (\text{young} \wedge f)$
Q.7	$\Diamond (\chi_F^d (\bigcirc^d \text{elder}) \wedge \chi_F^u (\text{elder} \wedge \neg f))$	Q.8	$\chi_F^u (\text{aliceLoc} == 1)$
Q.9	$\Diamond (\text{ret} \wedge \text{main} \wedge \text{aliceLoc} == 1)$	Q.10	$\neg \chi_F^u (\text{R} == 0 \wedge \text{C} == 2)$
Q.11	$\neg \chi_F^u (\text{R} == 0 \wedge \text{C} == 1)$	Q.12	$\neg \text{obs} \mathcal{U} (\text{call} \wedge \text{Bob} \wedge \chi_F^u \text{obs})$
Q.13	$\neg \text{Alice} \mathcal{U} (\text{call} \wedge \text{Alice} \wedge \neg \chi_F^u \text{obs})$	Q.14	$\Diamond (\text{ret} \wedge \text{main} \wedge \text{R} == 0 \wedge \text{C} == 2)$
Q.15	$\neg \text{obs} \mathcal{U} (\text{call} \wedge \text{Alice} \wedge \chi_F^u \text{obs})$		
Q.16	$\neg \text{sampleA} \mathcal{U} (\text{call} \wedge \text{sampleA} \wedge \chi_F^d (\text{qry} \wedge \bigcirc^d (\text{call} \wedge \text{sampleA} \wedge \chi_F^d (\text{qry} \wedge \bigcirc^d (\text{call} \wedge \text{sampleA}))))))$		
S.1	$\Box (\text{call} \wedge \text{B} \wedge \text{sorted} \wedge \text{valOccurs} \wedge \text{left} \leq \text{right} \implies \chi_F^u \text{a}[\text{mid}] == \text{val})$		
S.2	$\Box (\text{call} \wedge \text{B} \wedge \text{sorted} \wedge \neg \text{valOccurs} \wedge \text{left} \leq \text{right} \implies \chi_F^u \text{a}[\text{mid}] \neq \text{val})$		
S.3	$\Box (\text{call} \wedge \text{B} \wedge \text{sorted} \wedge \text{valOccurs} \wedge \text{left} < \text{right} \implies \bigcirc^d \Diamond (\text{call} \wedge \text{B}))$		
S.4	$\Box (\text{call} \wedge \text{B} \wedge \text{sorted} \wedge \neg \text{valOccurs} \wedge \text{left} < \text{right} \implies \bigcirc^d \Diamond (\text{call} \wedge \text{B}))$		

5 Evaluation

We devised our experiments around the following questions: can our approach build support chains for probabilistic programs of medium size? How large equations systems arise in quantitative model checking? How scalable is our implementation, i.e., how large programs can our tool solve in a reasonable time?

Setup. Our benchmark consists of three programs (*Schelling*, *Tic-tac-toe* and *Virus*) and 16 mixed LTL/POTL χ formulae (Table 1, **Q** formulae). All programs consist of a potentially unbounded sequence of nested queries. This places them at the frontier of probabilistic programming: even solving simple questions about their posterior distribution is a hardly tractable problem [42]. Additionally, to inquire scalability, we provide a case study on the *Sherwood* binary search with increasing program state space, against four formulae (Table 1, **S** formulae). We run the experiments on a machine with a 4.5GHz 8-core AMD CPU and 64 GB of RAM running Ubuntu 24.04. We do not offer an experimental comparison with PReMo [53] nor Pray [51], since they only compute termination probabilities on pPDA. This task is not the bottleneck of the overall model checking algorithm—our tool always computes them in less than one second.

We describe in the following the three programs, (some of) the formulae we verify on them, and their results. Afterwards, we present the case study separately. Two formulae inspecting conditioning are of interest for all three (**Q.1** and **Q.2**). If an observe statement conditions on an event with zero probability, all runs of a query are rejected. Such *ill-defined* queries [29] do not represent a valid probability distribution, which is undesired. LTL property **Q.1** means that a run is always sooner or later reinstated into a feasible one. However, if queries indefinitely call each other in a nonterminating program, a diverging run

indefinitely hits unsatisfied observations in the infinite nesting. POTLfx formula **Q.2** means that no procedure is indefinitely instantiated by an observation: either it terminates, or it diverges.

Schelling (Fig. 7). We consider an instance of a *Schelling coordination game* [44, 48]. Two agents wish to meet in town but cannot communicate. However, they know perfectly each other’s preferences. Each agent samples both a location according to its preference, and one obtained simulating the behavior of the other agent. Finally, it conditions on the two being equal. The original program [48] bounds the recursion depth by a constant. Conversely, we allow for unbounded recursion, but we introduce a global variable p controlling the recursion probability. We fix p ’s distribution so that the program is AST. Formula **Q.13** is the event that the first call to **Bob** is rejected, while **Q.14** that the first call to **Alice** is *not* rejected. Since Bob is not always fair, his procedure is much less likely to be rejected.

```

u4 p;
main() {
  bool res;
  p = 0{2:6}1{1:6}2{1:6}3{1:6}4;
  query alice(res);
}
Alice(bool &res) {
  aliceLoc = Bernoulli(55,100);
  query Bob(bobLoc);
  observe (aliceLoc == bobLoc);
  res = aliceLoc;
}
Bob(bool &res) {
  bobLoc = Bernoulli(55,100);
  fair = true {p:10} false;
  if (fair) {
    query Alice(aliceLoc);
    observe (bobLoc == aliceLoc);
  }
  res = bobLoc;
}

```

Fig. 7. Schelling.

Tic-tac-toe. Recursive probabilistic programs also model multi-player games involving sequential decision-making, i.e., where the best move in a turn depends on moves in subsequent turns. In these games, players choose actions

that maximize future rewards by simulating, through nested queries, other players’ turns as the game progresses. We consider tic-tac-toe [48]. A procedure modelling a player’s reasoning first marks uniformly at random a cell between those not taken yet; then it recursively queries itself to simulate the other player’s behaviour with the updated grid. This way, the whole game is recursively simulated, ending in a draw or in one of the two players winning. Finally, it conditions on a coin flip with weight corresponding to the utility of the outcome, thus performing softmax optimal decision making. We inspect the max/min number of turns until termination of the game, corresponding to the program’s max/min recursion depth. **Q.4** means that there are at least two recursive calls (i.e., two turns) —what we would expect from the initial state of Fig. 8, where no move can immediately terminate the game. We also examine the players’ meta-reasoning: **Q.5** excludes that a player ever thinks that the other one will pick cell [0, 1] in the next turn, which is indeed already marked at the beginning.

		○	
○	×	×	
	○		

Fig. 8. [48]

Virus (Fig. 9). Recursive probabilistic programs encode epidemiological models and general multi-type Branching Processes [17, 27] as models of population dynamics in biology. They consist of an unbounded population of susceptible individuals belonging to distinct species. In the programs, the stack models the individuals currently involved in the system, and different procedures model different species’ behaviors. We consider the program from [50, Fig. 1]: a virus outbreak. Our population is composed of either young or elder individuals. We introduce conditioning in the model expressing that young ones are less likely to transmit the virus. **Q.6** encodes the CaRet formula of [50, p. 3]: a chain of infections of young people leads to the death of an elder person.

In $\text{POTL}f_\chi, (\neg \text{elder})\mathcal{U}_\chi^d(\text{young} \wedge f)$, where f is a boolean variable representing an elder person passing away. Young people cannot pass away in our model. Note here the need for the χ variant of the $\mathcal{U}$ operator to capture exactly the property. Consider the case of a young person infecting first an elder person who survives, and then an elder who passes away. There would be only one young person in the chain of infections, however the LTL $(\neg \text{elder})\mathcal{U}(\text{young} \wedge f)$ would not hold: the first elder’s infection happens temporally before the death, although not part of the chain. $\text{POTL}f_\chi$ instead captures this context-free property: $\mathcal{U}_\chi^d$ exploits the χ relation to skip the first call to `elder()`—this example trace would satisfy it. **Q.7** means that eventually an elder person infects at least one elder person, but none of them passes away.

```

young() {
  bool f;
  ... (other statements)
  ... (sample infecting young)
  if (infect_young) {
    query young();
  }
  ... (sample infecting elders)
  if (infect_elder) {
    query elder(&f);
    query elder(&f);
  }
}

elder(bool &f) {
  ... (same as young())
  ... (sample passing away)
  if (pass_away) {
    f = 1;
  } else { f = 0; }
}

```

Fig. 9. Virus (sketch).

Results. We report the experimental results for building the support chain (Table 3, cf. Fig. 4) and for solving qualitative and quantitative model checking (Table 4, cf. Fig. 5). All times are in seconds and the timeout is 1 h. A key metric is the number of equations arising from each instance. Since we decompose the graph into SCCs and solve it bottom-up, many equations are solved just by propagating already computed values. We thus report how many equations actually require running our semi-algorithm because they introduce circular dependency between variables in the system. We call them **non-trivial** (NT) equations. In Table 3, $|Q_A|, |f|_{(\mathcal{A})}, |f_{NT}|_{(\mathcal{A})}$ are the number of resp. states, equations and non-trivial equations that arise when analysing pOPA $\mathcal{A}$ modeling the program. To estimate the size of individual SCCs we feed to OVI/Z3, we report also $|f_{NT}(\text{SCC})|_{\max(\mathcal{A})}$, the maximum number of non-trivial equations in an SCC of the equation system of $\mathcal{A}$. In Table 4, AS tells whether the formula holds almost surely. If it is not the case, we perform quantitative model checking to get probability P, for which we provide the same metrics as above, but on $\hat{\mathcal{A}}$, the cross-product between $\mathcal{A}$ and the formula automaton (Sect. 3.2).

Sherwood. The pOPA state space is ruled by two parameters: K, the number of bits of each array element, and M, the array’s size. We investigate $K \in [1, 4]$ and

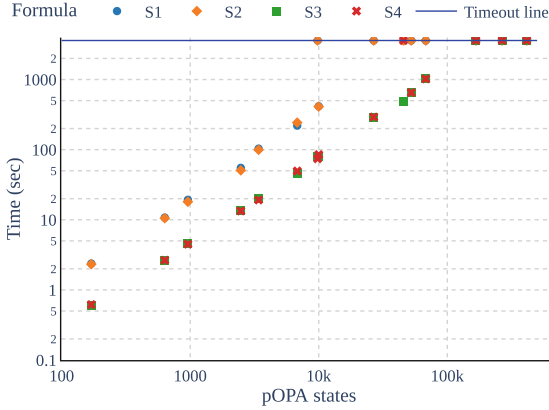


Fig. 10. Sherwood binary search: qualitative model checking of all formulae (both axis are on logarithmic scale).

Table 2. Sherwood binary search: quantitative model checking of **S.3** for some values of K and M .

		Quantitative MC		
K	M	$ f _{(\mathcal{A})}$	t_{tot}	P
1	1	154k	1.18	1
1	2	2 M	12.12	~ 0.906
1	3	8 M	53.87	~ 0.882
2	1	598k	5.20	1
2	2	15 M	111.36	~ 0.961
3	1	2 M	23.24	1
4	1	9 M	123.63	1

Table 3. Experimental results for the computation of termination probabilities. t_{Z3} (resp. t_{OVI}) is the time required to compute upper bounds with Z3 (OVI).

name	$ Q_{\mathcal{A}} $	$ f _{(\mathcal{A})}$	$ f_{NT} _{(\mathcal{A})}$	$ f_{NT}(\text{SCC}) _{\max(\mathcal{A})}$	t_{Z3}	t_{OVI}	t_{PAST}
Schelling	311	1230	266	52	TO	0.03	0.03
Tic-tac-toe	1780	3443	248	36	0.02	0.01	0.08
Virus	427	21211	6490	6488	TO	0.38	0

$M \in [1, 7]$. For each instance, the main procedure randomly initializes the array and a value to search, and calls **B()**. We assume that calls to **uniform()** diverge if **left** > **right**. Formula **S.1** is partial correctness from Sect. 1. Formula **S.2** is a dual version. Formulae **S.3** and **S.4** are stack-inspection properties: if **B()** is invoked in a state where **left** $\leq$ **right**, **a[*left*..*right*]** is sorted and **val** occurs in **a[*left*..*right*]** (does not, in **S.4**), then there is a recursive call to **B()**. All formulae hold almost surely except **S.3**: if the array has multiple elements, the randomized pivot selection may pick the searched value, terminating the program immediately. Figure 10 and Table 2 show results for qualitative model checking on all formulae, and quantitative for some parameters of **S.3**.

Table 4. Qualitative and quantitative MC. t_G , t_{OVI} are times spent resp. analyzing the synchronized product and computing with OVI upper bounds for non trivial equations.

name	φ	Qualitative MC			Quantitative MC					
		t_G	t_{tot}	AS	$ f _{(\bar{A})}$	$ f_{NT} _{(\bar{A})}$	$ f_{NT}(\text{SCC}) _{\max(\bar{A})}$	t_{OVI}	t_{tot}	P
Schelling	Q.1	0.02	0.15	✓	-	-	-	-	-	1
	Q.2	26.28	26.40	✓	-	-	-	-	-	1
	Q.3	1.25	1.37	✗	893k	68k	7.3k	11.71	20.58	~ 0.895
	Q.8	0.22	0.34	✗	214k	15k	1.4k	3.80	5.70	~ 0.610
	Q.9	0.04	0.16	✗	36k	3.2k	252	1.29	1.57	~ 0.610
	Q.12	1.01	1.14	✗	651k	61k	6.7k	11.56	20.10	~ 0.096
	Q.13	0.67	0.80	✗	599k	43k	5.7k	6.48	13.28	~ 0.506
	Q.15	0.95	1.07	✗	762k	63k	7.5k	12.49	23.21	~ 0.543
Tic-tac-toe	Q.1	0.12	0.2	✓	-	-	-	-	-	1
	Q.2	122.16	122.33	✓	-	-	-	-	-	1
	Q.4	9.79	9.98	✓	-	-	-	-	-	1
	Q.5	8.22	8.41	✓	-	-	-	-	-	1
	Q.10	2.94	3.10	✗	1.1 M	77k	3k	0.79	10.45	~ 0.712
	Q.11	2.96	3.13	✓	-	-	-	-	-	1
	Q.14	0.54	0.71	✗	280k	20k	1.5k	0.17	2.30	~ 0.288
	Q.16	386.10	386.22	✓	-	-	-	-	-	1
Virus	Q.1	0.18	2.13	✗	243k	25k	12k	0.50	9.72	~ 0.239
	Q.2	263.94	265.9	✓	-	-	-	-	-	1
	Q.6	14.96	16.95	✗	?	?	?	?	?	TO
	Q.7	891.45	893.37	✗	?	?	?	?	?	TO

6 Discussion and Future Work

Experimental results show that OVI scales better than Z3 in solving systems for building the support chain. Z3 already times out when provided with a system of 52 equations. For quantitative model checking, the exponential blow-up on the formula automaton generates in our benchmark hundred of thousands of equations. This suggests that determinization-based approaches, leading to another exponentiation, would not work. Finally, our tool scales up to a few million of equations, supporting checking non-trivial formulae on medium-size pPDA.

Future improvements may come from finding ways of modeling non-trivial probabilistic programs with less expressive pPDA subclasses that have more efficient model checking algorithms. pPDA with one single state (or *stochastic context-free grammars*) admit P-time algorithms for computing the probability of termination [15]; *probabilistic one-counter automata* admit P-time algorithms for model checking ω -regular properties [47].

Acknowledgments. We thank Tobias Winkler and Joost-Pieter Katoen (RWTH Aachen) for the fruitful discussions and advice on implementing OVI. This work was funded by the Vienna Science and Technology Fund (WWTF) grants [10.47379/ICT19018] (ProbInG) and ICT22-023 (TAIGER), and by the Horizon Europe programme grants No. 101034440 (MSCA Doctoral Network Log-iCS@TU Wien) and No. 101107303 (MSCA Postdoctoral Fellowship CORPORA) ■.

Disclosure of Interests. The authors declare no competing interests for this article.

References

1. Baier, C., Katoen, J.: Principles of Model Checking. MIT Press (2008)
2. Baier, C., Kiefer, S., Klein, J., Müller, D., Worrell, J.: Markov chains and unambiguous automata. *J. Comput. Syst. Sci.* **136**, 113–134 (2023). <https://doi.org/10.1016/J.JCSS.2023.03.005>
3. Brázdil, T., Brozek, V., Holecek, J., Kucera, A.: Discounted properties of probabilistic pushdown automata. In: LPAR 2008. LNCS, vol. 5330, pp. 230–242. Springer (2008). https://doi.org/10.1007/978-3-540-89439-1_17
4. Brázdil, T., Esparza, J., Kiefer, S., Kucera, A.: Analyzing probabilistic pushdown automata. *Formal Methods Syst. Des.* **43**(2), 124–163 (2013). <https://doi.org/10.1007/s10703-012-0166-0>
5. Brázdil, T., Esparza, J., Kucera, A.: Analysis and prediction of the long-run behavior of probabilistic sequential programs with recursion (extended abstract). In: FOCS 2005, pp. 521–530. IEEE Computer Society (2005). <https://doi.org/10.1109/SFCS.2005.19>
6. Brázdil, T., Kiefer, S., Kucera, A., Vareková, I.H.: Runtime analysis of probabilistic programs with unbounded recursion. *J. Comput. Syst. Sci.* **81**(1), 288–310 (2015). <https://doi.org/10.1016/J.JCSS.2014.06.005>
7. Canny, J.F.: Some algebraic and geometric computations in PSPACE. In: STOC 1988, pp. 460–467. ACM (1988). <https://doi.org/10.1145/62212.62257>
8. Chiari, M., Mandrioli, D., Pontiggia, F., Pradella, M.: A model checker for operator precedence languages. *ACM Trans. Program. Lang. Syst.* **45**(3) (2023). <https://doi.org/10.1145/3608443>
9. Chiari, M., Mandrioli, D., Pradella, M.: Model-checking structured context-free languages. In: Silva, A., Leino, K. (eds.) CAV 2021. LNCS, vol. 12760, pp. 387–410. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-81688-9_18
10. Chiari, M., Mandrioli, D., Pradella, M.: A first-order complete temporal logic for structured context-free languages. *Log. Methods Comput. Sci.* **18**:3 (2022). [https://doi.org/10.46298/LMCS-18\(3:11\)2022](https://doi.org/10.46298/LMCS-18(3:11)2022)
11. Couvreur, J., Saheb, N., Sutre, G.: An optimal automata approach to LTL model checking of probabilistic systems. In: LPAR 2003. LNCS, vol. 2850, pp. 361–375. Springer (2003). https://doi.org/10.1007/978-3-540-39813-4_26
12. Esparza, J., Kiefer, S., Luttenberger, M.: Computing the least fixed point of positive polynomial systems. *SIAM J. Comput.* **39**(6), 2282–2335 (2010). <https://doi.org/10.1137/090749591>
13. Esparza, J., Kretínský, J., Sickert, S.: A unified translation of linear temporal logic to ω -automata. *J. ACM* **67**(6), 33:1–33:61 (2020). <https://doi.org/10.1145/3417995>

14. Esparza, J., Kucera, A., Mayr, R.: Model checking probabilistic pushdown automata. In: LICS 2004, pp. 12–21. IEEE Computer Society (2004). <https://doi.org/10.1109/LICS.2004.1319596>
15. Etessami, K., Stewart, A., Yannakakis, M.: Polynomial time algorithms for multi-type branching processes and stochastic context-free grammars. In: STOC 2012, pp. 579–588. ACM (2012). <https://doi.org/10.1145/2213977.2214030>
16. Etessami, K., Yannakakis, M.: Algorithmic verification of recursive probabilistic state machines. In: TACAS 2005. LNCS, vol. 3440, pp. 253–270. Springer (2005). https://doi.org/10.1007/978-3-540-31980-1_17
17. Etessami, K., Yannakakis, M.: Recursive Markov chains, stochastic grammars, and monotone systems of nonlinear equations. In: STACS 2005. LNCS, vol. 3404, pp. 340–352. Springer (2005). https://doi.org/10.1007/978-3-540-31856-9_28
18. Etessami, K., Yannakakis, M.: Recursive Markov chains, stochastic grammars, and monotone systems of nonlinear equations. J. ACM **56**(1), 1:1–1:66 (2009). <https://doi.org/10.1145/1462153.1462154>
19. Etessami, K., Yannakakis, M.: Model checking of recursive probabilistic systems. ACM Trans. Comput. Log. **13**(2), 12:1–12:40 (2012). <https://doi.org/10.1145/2159531.2159534>
20. Etessami, K., Yannakakis, M.: Recursive Markov decision processes and recursive stochastic games. J. ACM **62**(2), 11:1–11:69 (2015). <https://doi.org/10.1145/2699431>
21. Evans, O., Stuhlmüller, A., Salvatier, J., Filan, D.: Modeling agents with probabilistic programs (2017). <http://agentmodels.org>
22. Floyd, R.W.: Syntactic analysis and operator precedence. J. ACM **10**(3), 316–333 (1963). <https://doi.org/10.1145/321172.321179>
23. Ghahramani, Z.: Probabilistic machine learning and artificial intelligence. Nat. **521**(7553), 452–459 (2015). <https://doi.org/10.1038/NATURE14541>
24. Goodman, N.D., Mansinghka, V.K., Roy, D.M., Bonawitz, K.A., Tenenbaum, J.B.: Church: a language for generative models. In: UAI 2008, pp. 220–229. AUAI Press (2008)
25. Goodman, N.D., Tenenbaum, J.B., The ProbMods contributors: probabilistic models of cognition. <http://probmods.org/v2> (2016). Accessed 22 May 2025
26. Gordon, A.D., Henzinger, T.A., Nori, A.V., Rajamani, S.K.: Probabilistic programming. In: FOSE 2014, pp. 167–181. ACM (2014). <https://doi.org/10.1145/2593882.2593900>
27. Haccou, P., Jagers, P., Vatutin, V.A.: Branching Processes: Variation, Growth, and Extinction of Populations. Cambridge University Press (2005)
28. Hensel, C., Junges, S., Katoen, J., Quatmann, T., Volk, M.: The probabilistic model checker Storm. Int. J. Softw. Tools Technol. Transf. **24**(4), 589–610 (2022). <https://doi.org/10.1007/s10009-021-00633-z>
29. Jacobs, J.: Paradoxes of probabilistic programming: and how to condition on events of measure zero with infinitesimal probabilities. ACM Program. Lang. **5**(POPL), 1–26 (2021). <https://doi.org/10.1145/3434339>
30. Jovanovic, D., de Moura, L.: Solving non-linear arithmetic. ACM Commun. Comput. Algebra **46**(3/4), 104–105 (2012). <https://doi.org/10.1145/2429135.2429155>
31. Kucera, A., Esparza, J., Mayr, R.: Model checking probabilistic pushdown automata. Log. Methods Comput. Sci. **2**(1) (2006). [https://doi.org/10.2168/LMCS-2\(1:2\)2006](https://doi.org/10.2168/LMCS-2(1:2)2006)
32. Kwiatkowska, M.Z., Norman, G., Parker, D.: PRISM 4.0: verification of probabilistic real-time systems. In: CAV 2011. LNCS, vol. 6806, pp. 585–591. Springer (2011). https://doi.org/10.1007/978-3-642-22110-1_47

33. Mandrioli, D., Pradella, M.: Generalizing input-driven languages: theoretical and practical benefits. *Comput. Sci. Rev.* **27**, 61–87 (2018). <https://doi.org/10.1016/j.cosrev.2017.12.001>
34. McConnell, J.: *Analysis of Algorithms*. Jones & Bartlett Publishers (2007)
35. de Moura, L.M., Bjørner, N.S.: Z3: an efficient SMT solver. In: TACAS 2008. LNCS, vol. 4963, pp. 337–340. Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_24
36. Olmedo, F., Gretz, F., Jansen, N., Kaminski, B.L., Katoen, J., McIver, A.: Conditioning in probabilistic programming. *ACM Trans. Program. Lang. Syst.* **40**(1), 4:1–4:50 (2018). <https://doi.org/10.1145/3156018>
37. Olmedo, F., Kaminski, B.L., Katoen, J., Matheja, C.: Reasoning about recursive probabilistic programs. In: LICS 2016, pp. 672–681. ACM (2016). <https://doi.org/10.1145/2933575.2935317>
38. Pnueli, A.: The temporal logic of programs. In: FOCS 1977, pp. 46–57. IEEE Computer Society (1977). <https://doi.org/10.1109/SFCS.1977.32>
39. Pontiggia, F.: POMC. A model checking tool for operator precedence languages on omega-words. Master’s thesis, Politecnico di Milano (2021). <http://hdl.handle.net/10589/176028>
40. Pontiggia, F., Bartocci, E., Chiari, M.: Model checking probabilistic operator precedence automata. *CoRR* (2024). <https://doi.org/10.48550/arXiv.2404.03515>
41. Pontiggia, F., Chiari, M., Pradella, M.: Verification of programs with exceptions through operator precedence automata. In: SEFM 2021. LNCS, vol. 13085, pp. 293–311. Springer, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-030-92124-8_17
42. Rainforth, T.: Nesting probabilistic programs. In: Globerson, A., Silva, R. (eds.) UAI 2018, pp. 249–258. AUAI Press (2018). <http://auai.org/uai2018/proceedings/papers/92.pdf>
43. Renegar, J.: On the computational complexity and geometry of the first-order theory of the reals, parts I–III. *J. Symb. Comput.* **13**(3), 255–352 (1992). [https://doi.org/10.1016/S0747-7171\(10\)80003-3](https://doi.org/10.1016/S0747-7171(10)80003-3)
44. Schelling, T.C.: *The Strategy of Conflict*. Harvard University Press (1980)
45. Scontras, G., Tessler, M.H., Franke, M.: Probabilistic language understanding: an introduction to the rational speech act framework. <https://www.problang.org/>. Accessed 22 May 2025
46. Seaman, I.R., van de Meent, J.W., Wingate, D.: Nested reasoning about autonomous agents using probabilistic programs (2020). <https://arxiv.org/abs/1812.01569>
47. Stewart, A., Etessami, K., Yannakakis, M.: Upper bounds for Newton’s method on monotone polynomial systems, and P-time model checking of probabilistic one-counter automata. *J. ACM* **62**(4), 30:1–30:33 (2015). <https://doi.org/10.1145/2789208>
48. Stuhlmüller, A., Goodman, N.D.: Reasoning about reasoning by nested conditioning: modeling theory of mind with probabilistic programs. *Cogn. Syst. Res.* **28**, 80–99 (2014). <https://doi.org/10.1016/J.COGSYS.2013.07.003>
49. Vardi, M.Y., Wolper, P.: Reasoning about infinite computations. *Inf. Comput.* **115**(1), 1–37 (1994). <https://doi.org/10.1006/INCO.1994.1092>
50. Winkler, T., Gehnen, C., Katoen, J.: Model checking temporal properties of recursive probabilistic programs. In: FOSSACS 2022. LNCS, vol. 13242, pp. 449–469. Springer (2022). https://doi.org/10.1007/978-3-030-99253-8_23

51. Winkler, T., Katoen, J.: Certificates for probabilistic pushdown automata via optimistic value iteration. In: TACAS 2023. LNCS, vol. 13994, pp. 391–409. Springer (2023). https://doi.org/10.1007/978-3-031-30820-8_24
52. Winkler, T., Katoen, J.: On certificates, expected runtimes, and termination in probabilistic pushdown automata. In: LICS 2023, pp. 1–13 (2023). <https://doi.org/10.1109/LICS56636.2023.10175714>
53. Wojtczak, D., Etessami, K.: PReMo: An analyzer for probabilistic recursive models. In: TACAS 2007. LNCS, vol. 4424, pp. 66–71. Springer (2007). https://doi.org/10.1007/978-3-540-71209-1_7
54. Yannakakis, M., Etessami, K.: Checking LTL properties of recursive Markov chains. In: QEST 2005, pp. 155–165. IEEE (2005). <https://doi.org/10.1109/QEST.2005.8>
55. Zhang, Y., Amin, N.: Reasoning about “reasoning about reasoning”: semantics and contextual equivalence for probabilistic programs with nested queries and recursion. *ACM Program. Lang.* **6**(POPL), 1–28 (2022). <https://doi.org/10.1145/3498677>

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter’s Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter’s Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

